Experiment 'p_4_2_d_adjacent_duplicates_array_without_4th_ensures' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 4
Hard fails (resolution): 1
Soft fails (resolution): 1
Verification attempts: 2

Problem Specification

Problem name: p_4_2_d_adjacent_duplicates_array_without_4th_ensures
Natural language statement: null
Method signature: p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (duplicates: array<int>)

Ensures

- `forall d :: d in duplicates[..] ==> d in inputs[..]`
- `forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..]`
- `forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] && inputs[i] == d`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (
    ↪ duplicates: array<int>)

The method should respect the following contract:

ensures forall d :: d in duplicates[..] ==> d in inputs[..], ensures forall i :: 0 <= i <
    ↪  inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..],
    ↪ ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
    ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLh4ZElLp4o77897TuNrSDZSchJO
Model: gpt-5.5-2026-04-23
Created at: 1780653834
Finish reason: stop
Usage: 434 tokens in, and 4788 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j < inputs.Length - 1 &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - 1 - i
  {
    var nextI := i + 1;

    if inputs[i] == inputs[i + 1] {
      var oldAcc := acc;
      acc := oldAcc + [inputs[i]];
```

```
  assert inputs[i] in acc;

  assert forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    if d in oldAcc {
      assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j
          ↪ + 1] && inputs[j] == d;
      assert forall j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j
          ↪ + 1] && inputs[j] == d
        ==> 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j + 1] &&
            ↪ inputs[j] == d;
    } else {
      assert d in [inputs[i]];
      assert d == inputs[i];
      assert 0 <= i < nextI;
      assert i < inputs.Length - 1;
      assert exists j :: 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] ==
          ↪ inputs[j + 1] && inputs[j] == d;
    }
  }

  assert forall j | 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j +
      ↪  1]
    ensures inputs[j] in acc
  {
    if j < i {
      assert inputs[j] in oldAcc;
      assert inputs[j] in acc;
    } else {
      assert j == i;
      assert inputs[j] == inputs[i];
      assert inputs[j] in acc;
    }
  }
} else {
  assert forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j +
        ↪ 1] && inputs[j] == d;
    assert forall j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j +
        ↪ 1] && inputs[j] == d
      ==> 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d;
  }

  assert forall j | 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j +
```

```
        ↪ 1]
      ensures inputs[j] in acc
    {
      if j < i {
        assert inputs[j] in acc;
      } else {
        assert j == i;
        assert inputs[i] != inputs[i + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
 }

 duplicates := new int[|acc|](j => acc[j]);
 assert duplicates.Length == |acc|;
 assert forall j :: 0 <= j < duplicates.Length ==> duplicates[j] == acc[j];
 assert duplicates[..] == acc;

 assert forall j :: 0 <= j < inputs.Length - 1 ==> j < i;

 assert forall d | d in duplicates[..]
   ensures d in inputs[..]
 {
   assert d in acc;
   assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
       ↪ && inputs[j] == d;
 }

 assert forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
   ensures inputs[j] in duplicates[..]
 {
   assert j < i;
   assert inputs[j] in acc;
 }

 assert forall d | d in duplicates[..]
   ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
       ↪ [j] == d
 {
   assert d in acc;
   assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
       ↪ && inputs[j] == d;
 }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(51,4):
    ↪ Error: invalid UnaryExpression
   |
51 | } else {
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(50,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
50 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(50,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
50 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(72,4):
    ↪ Error: invalid UnaryExpression
   |
72 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(71,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
71 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(71,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
71 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(104,0):
    ↪ Error: invalid UnaryExpression
    |
104 | }
    | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(103,2):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
    |
103 | }
    | ^
```

```
problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(103,2):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
    |
103 | }
    | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(103,2):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
    |
103 | }
    | ^

10 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLie4OE3lIevzRNRzhVZRgvqiyNW
Model: gpt-5.5-2026-04-23
Created at: 1780653932
Finish reason: stop
Usage: 2593 tokens in, and 6074 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - i
  {
    var oldI := i;
    var nextI := i + 1;

    if inputs[oldI] == inputs[oldI + 1] {
      var oldAcc := acc;
      assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
          ↪  1] ==> inputs[j] in oldAcc;
      assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
          ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

      acc := oldAcc + [inputs[oldI]];
```

```
  forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    assert d in oldAcc + [inputs[oldI]];
    assert d in oldAcc || d in [inputs[oldI]];
    if d in oldAcc {
      var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
          ↪  inputs[j] == d;
      assert 0 <= j < nextI;
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
          ↪ inputs[k + 1] && inputs[k] == d;
    } else {
      assert d in [inputs[oldI]];
      assert d == inputs[oldI];
      assert 0 <= oldI < nextI;
      assert oldI + 1 < inputs.Length;
      assert inputs[oldI] == inputs[oldI + 1];
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
          ↪ inputs[k + 1] && inputs[k] == d;
    }
  }

  forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
    ensures inputs[j] in acc
  {
    if j < oldI {
      assert inputs[j] in oldAcc;
      assert inputs[j] in acc;
    } else {
      assert j == oldI;
      assert inputs[j] == inputs[oldI];
      assert inputs[oldI] in [inputs[oldI]];
      assert inputs[oldI] in acc;
      assert inputs[j] in acc;
    }
  }
} else {
  forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
        ↪ inputs[j] == d;
    assert 0 <= j < nextI;
    assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
        ↪ k + 1] && inputs[k] == d;
  }

  forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
    ensures inputs[j] in acc
```

```
    {
      if j < oldI {
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[oldI] != inputs[oldI + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
}

duplicates := new int[|acc|](j => acc[j]);
assert duplicates.Length == |acc|;
assert |duplicates[..]| == |acc|;
forall j | 0 <= j < |acc|
  ensures duplicates[..][j] == acc[j]
{
  assert duplicates[..][j] == duplicates[j];
  assert duplicates[j] == acc[j];
}
assert duplicates[..] == acc;

forall j | 0 <= j < inputs.Length - 1
  ensures j < i
{
  assert !(i + 1 < inputs.Length);
  assert i + 1 >= inputs.Length;
  assert j + 1 < inputs.Length;
  assert j < i;
}

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in acc;
  var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
      ↪ ] == d;
  assert 0 <= j < inputs.Length;
  assert inputs[j] == d;
  assert inputs[j] in inputs[..];
}

forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates[..]
{
  assert j < i;
  assert j + 1 < inputs.Length;
```

```
    assert inputs[j] in acc;
    assert inputs[j] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
        ↪ [j] == d
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert j < inputs.Length - 1;
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(95,2):
    ↪ Warning: Could not find a trigger for this quantifier. Without a trigger, the
    ↪ quantifier may cause brittle verification. To silence this warning, add an
    ↪ explicit trigger using the {:trigger} attribute. For more information, see the
    ↪ section on quantifier instantiation rules in the reference manual.
   |
95 | forall j | 0 <= j < inputs.Length - 1
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLkX4x29QM7hK3MeRG9bHtLpEgYB
Model: gpt-5.5-2026-04-23
Created at: 1780654049
Finish reason: stop
Usage: 3902 tokens in, and 2336 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
```

```
  invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
      ↪ 1] ==> inputs[j] in acc
  invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
      ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
  decreases inputs.Length - i
{
  var oldI := i;
  var nextI := i + 1;

  if inputs[oldI] == inputs[oldI + 1] {
    var oldAcc := acc;
    assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪  1] ==> inputs[j] in oldAcc;
    assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
        ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

    acc := oldAcc + [inputs[oldI]];

    forall d | d in acc
      ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
          ↪ [j + 1] && inputs[j] == d
    {
      assert d in oldAcc + [inputs[oldI]];
      assert d in oldAcc || d in [inputs[oldI]];
      if d in oldAcc {
        var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
            ↪  inputs[j] == d;
        assert 0 <= j < nextI;
        assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
            ↪ inputs[k + 1] && inputs[k] == d;
      } else {
        assert d in [inputs[oldI]];
        assert d == inputs[oldI];
        assert 0 <= oldI < nextI;
        assert oldI + 1 < inputs.Length;
        assert inputs[oldI] == inputs[oldI + 1];
        assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
            ↪ inputs[k + 1] && inputs[k] == d;
      }
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in oldAcc;
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[j] == inputs[oldI];
        assert inputs[oldI] in [inputs[oldI]];
```

```
      assert inputs[oldI] in acc;
      assert inputs[j] in acc;
    }
  }
} else {
  forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
        ↪ inputs[j] == d;
    assert 0 <= j < nextI;
    assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
        ↪ k + 1] && inputs[k] == d;
  }

  forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
    ensures inputs[j] in acc
  {
    if j < oldI {
      assert inputs[j] in acc;
    } else {
      assert j == oldI;
      assert inputs[oldI] != inputs[oldI + 1];
      assert inputs[j] == inputs[j + 1];
      assert false;
    }
  }
}

i := nextI;
}

duplicates := new int[|acc|](j => acc[j]);
assert duplicates.Length == |acc|;
assert |duplicates[..]| == |acc|;
forall j | 0 <= j < |acc|
  ensures duplicates[..][j] == acc[j]
{
  assert duplicates[..][j] == duplicates[j];
  assert duplicates[j] == acc[j];
}
assert duplicates[..] == acc;

assert !(i + 1 < inputs.Length);
assert i + 1 >= inputs.Length;

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in acc;
```

```
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert 0 <= j < inputs.Length;
    assert inputs[j] == d;
    assert inputs[j] in inputs[..];
  }

  forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates[..]
  {
    assert j + 1 < inputs.Length;
    assert j + 1 < i + 1;
    assert j < i;
    assert inputs[j] in acc;
    assert inputs[j] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
        ↪ [j] == d
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert j < inputs.Length - 1;
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(84,39):
    ↪ Error: index out of range
   |
84 | duplicates := new int[|acc|](j => acc[j]);
   |               ^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLlWG9hoUWpQV1DTNUYGOqwXzVxu

Model: gpt-5.5-2026-04-23
Created at: 1780654110
Finish reason: stop
Usage: 5460 tokens in, and 2541 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - i
  {
    var oldI := i;
    var nextI := i + 1;

    if inputs[oldI] == inputs[oldI + 1] {
      var oldAcc := acc;
      assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
          ↪  1] ==> inputs[j] in oldAcc;
      assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
          ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

      acc := oldAcc + [inputs[oldI]];

      forall d | d in acc
        ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
            ↪ [j + 1] && inputs[j] == d
      {
        assert d in oldAcc + [inputs[oldI]];
        assert d in oldAcc || d in [inputs[oldI]];
        if d in oldAcc {
          var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
              ↪  inputs[j] == d;
          assert 0 <= j < nextI;
          assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
              ↪ inputs[k + 1] && inputs[k] == d;
        } else {
          assert d in [inputs[oldI]];
          assert d == inputs[oldI];
          assert 0 <= oldI < nextI;
          assert oldI + 1 < inputs.Length;
          assert inputs[oldI] == inputs[oldI + 1];
          assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
              ↪ inputs[k + 1] && inputs[k] == d;
        }
      }
```

```
    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in oldAcc;
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[j] == inputs[oldI];
        assert inputs[oldI] in [inputs[oldI]];
        assert inputs[oldI] in acc;
        assert inputs[j] in acc;
      }
    }
  } else {
    forall d | d in acc
      ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
          ↪ [j + 1] && inputs[j] == d
    {
      var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d;
      assert 0 <= j < nextI;
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
          ↪ k + 1] && inputs[k] == d;
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[oldI] != inputs[oldI + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
}

duplicates := new int[|acc|];
var k := 0;
while k < |acc|
  invariant 0 <= k <= |acc|
  invariant duplicates.Length == |acc|
  invariant forall t :: 0 <= t < k ==> duplicates[t] == acc[t]
  decreases |acc| - k
{
```

```
    duplicates[k] := acc[k];
    k := k + 1;
  }

  assert duplicates.Length == |acc|;
  assert |duplicates[..]| == |acc|;
  forall t | 0 <= t < |acc|
    ensures duplicates[..][t] == acc[t]
  {
    assert duplicates[..][t] == duplicates[t];
    assert duplicates[t] == acc[t];
  }
  assert duplicates[..] == acc;

  assert !(i + 1 < inputs.Length);
  assert i + 1 >= inputs.Length;

  forall d | d in duplicates[..]
    ensures d in inputs[..]
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert 0 <= j < inputs.Length;
    assert inputs[j] == d;
    assert inputs[j] in inputs[..];
  }

  forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates[..]
  {
    assert j + 1 < inputs.Length;
    assert j + 1 < i + 1;
    assert j < i;
    assert inputs[j] in acc;
    assert inputs[j] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
        ↪ [j] == d
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert j < inputs.Length - 1;
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Final Program

```
method p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns
    ↪ (duplicates: array<int>)
      ensures forall d :: d in duplicates[..] ==> d in inputs[..]
      ensures forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==>
          ↪ inputs[i] in duplicates[..]
      ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
          ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - i
  {
    var oldI := i;
    var nextI := i + 1;

    if inputs[oldI] == inputs[oldI + 1] {
      var oldAcc := acc;
      assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
          ↪  1] ==> inputs[j] in oldAcc;
      assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
          ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

      acc := oldAcc + [inputs[oldI]];

      forall d | d in acc
        ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
            ↪ [j + 1] && inputs[j] == d
      {
        assert d in oldAcc + [inputs[oldI]];
        assert d in oldAcc || d in [inputs[oldI]];
        if d in oldAcc {
          var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
              ↪  inputs[j] == d;
          assert 0 <= j < nextI;
          assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
              ↪ inputs[k + 1] && inputs[k] == d;
        } else {
          assert d in [inputs[oldI]];
          assert d == inputs[oldI];
          assert 0 <= oldI < nextI;
          assert oldI + 1 < inputs.Length;
          assert inputs[oldI] == inputs[oldI + 1];
```

```
        assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
            ↪ inputs[k + 1] && inputs[k] == d;
      }
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in oldAcc;
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[j] == inputs[oldI];
        assert inputs[oldI] in [inputs[oldI]];
        assert inputs[oldI] in acc;
        assert inputs[j] in acc;
      }
    }
  } else {
    forall d | d in acc
      ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
          ↪ [j + 1] && inputs[j] == d
    {
      var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d;
      assert 0 <= j < nextI;
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
          ↪ k + 1] && inputs[k] == d;
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[oldI] != inputs[oldI + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
}

duplicates := new int[|acc|];
var k := 0;
while k < |acc|
```

```
  invariant 0 <= k <= |acc|
  invariant duplicates.Length == |acc|
  invariant forall t :: 0 <= t < k ==> duplicates[t] == acc[t]
  decreases |acc| - k
{
  duplicates[k] := acc[k];
  k := k + 1;
}

assert duplicates.Length == |acc|;
assert |duplicates[..]| == |acc|;
forall t | 0 <= t < |acc|
  ensures duplicates[..][t] == acc[t]
{
  assert duplicates[..][t] == duplicates[t];
  assert duplicates[t] == acc[t];
}
assert duplicates[..] == acc;

assert !(i + 1 < inputs.Length);
assert i + 1 >= inputs.Length;

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in acc;
  var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
      ↪ ] == d;
  assert 0 <= j < inputs.Length;
  assert inputs[j] == d;
  assert inputs[j] in inputs[..];
}

forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates[..]
{
  assert j + 1 < inputs.Length;
  assert j + 1 < i + 1;
  assert j < i;
  assert inputs[j] in acc;
  assert inputs[j] in duplicates[..];
}

forall d | d in duplicates[..]
  ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
      ↪ [j] == d
{
  assert d in acc;
  var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
      ↪ ] == d;
  assert j < inputs.Length - 1;
```

```
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Total Token Usage

Input tokens: 12389
Output tokens: 15739
Reasoning tokens: 9934
Sum of 'total tokens': 28128

Experiment Timings

Overall Experiment started at 1780653833903, ended at 1780654169144, lasting 335241ms (335.24 seconds)
Iteration #4 started at 1780654109033, ended at 1780654169143, lasting 60110ms (60.11 seconds)
Iteration #1 started at 1780653833903, ended at 1780653931544, lasting 97641ms (97.64 seconds)
Iteration #2 started at 1780653931544, ended at 1780654048206, lasting 116662ms (116.66 seconds)
Iteration #3 started at 1780654048207, ended at 1780654109033, lasting 60826ms (60.83 seconds)